

Sistema de Telemetria Espacial

Relatório acadêmico - verificação simplificada de condições para decolagem

Autor

Pedro Vinicius de Castro

Observação: as faixas de segurança e os valores utilizados são fictícios e servem somente para fins acadêmicos.

1. Introdução

Este projeto simula um sistema simples de análise de telemetria de uma missão espacial. O objetivo é organizar os dados, verificar limites predefinidos, decidir se a missão está pronta para decolar, calcular a autonomia energética e apresentar uma análise complementar assistida por IA.

1.1 Organização e descrição da telemetria

Parâmetro	Valor	Faixa segura simulada	Situação
Temperatura interna	24,5 °C	18 a 30 °C	Normal
Temperatura externa	42 °C	-20 a 50 °C	Normal
Integridade estrutural	1	Deve ser 1	Normal
Energia	92%	≥ 80%	Normal
Pressão dos tanques	305 bar	250 a 320 bar	Normal
Módulos críticos	Todos OK	Todos devem estar OK	Normal

A integridade estrutural é binária: 1 representa condição íntegra e 0 representa falha. Os módulos críticos considerados são propulsão, navegação, comunicação e suporte à vida.

1.2 Algoritmo de verificação

```
INICIO
  ler dados de telemetria
  criar lista de falhas
  verificar temperaturas
  verificar integridade estrutural
  verificar energia
  verificar pressao dos tanques
  verificar modulos criticos

  se lista de falhas estiver vazia:
    exibir "PRONTO PARA DECOLAR"
  senao:
    exibir "DECOLAGEM ABORTADA"
    exibir falhas encontradas
FIM
```

1.3 Script em Python

```
telemetria = {
    "temperatura_interna": 24.5,
    "temperatura_externa": 42.0,
    "integridade_estrutural": 1,
    "energia": 92,
    "pressao_tanques": 305,
    "modulos": {
        "propulsao": "OK",
        "navegacao": "OK",
        "comunicacao": "OK",
        "suporte_vida": "OK"
    }
}

def verificar_telemetria(dados):
    falhas = []

    if not 18 <= dados["temperatura_interna"] <= 30:
        falhas.append("Temperatura interna fora da faixa segura")
    if not -20 <= dados["temperatura_externa"] <= 50:
        falhas.append("Temperatura externa fora da faixa segura")
    if dados["integridade_estrutural"] != 1:
        falhas.append("Falha na integridade estrutural")
    if dados["energia"] < 80:
        falhas.append("Energia abaixo do minimo")
    if not 250 <= dados["pressao_tanques"] <= 320:
        falhas.append("Pressao fora da faixa segura")

    for modulo, status in dados["modulos"].items():
        if status != "OK":
            falhas.append(f"Falha no modulo: {modulo}")

    return falhas

falhas = verificar_telemetria(telemetria)
print("PRONTO PARA DECOLAR" if not falhas else "DECOLAGEM ABORTADA")
```

No cenário inicial, todos os parâmetros estão dentro das regras definidas e o resultado é PRONTO PARA DECOLAR. O notebook também contém um teste com energia em 60%, que retorna DECOLAGEM ABORTADA.

Print da execução

Execucao do notebook - Telemetria Espacial

```
=== SISTEMA DE TELEMETRIA ESPACIAL ===
Temperatura interna: 24.5 C
Temperatura externa: 42.0 C
Integridade estrutural: 1
Energia: 92%
Pressao dos tanques: 305 bar
Modulos:
- propulsao: OK
- navegacao: OK
- comunicacao: OK
- suporte_vida: OK
```

RESULTADO: PRONTO PARA DECOLAR

Saida gerada a partir do notebook executado.

1.4 Análise energética

Foram considerados: capacidade total de 500 kWh, carga atual de 92%, perdas de 8%, consumo de 120 kWh na decolagem e consumo médio de 35 kW após a decolagem.

Etapa	Cálculo	Resultado
Energia armazenada	$500 \times 0,92$	460,0 kWh
Perdas	$460 \times 0,08$	36,8 kWh
Energia útil	$460 - 36,8$	423,2 kWh
Após decolagem	$423,2 - 120$	303,2 kWh
Autonomia	$303,2 / 35$	8,66 horas

Assim, restam 303,2 kWh após a decolagem. Para um consumo médio de 35 kW, a autonomia estimada é de aproximadamente 8,66 horas.

1.5 Análise assistida por IA

Parâmetro	Classificação	Observação
Temperatura interna	Normal	Dentro da faixa
Temperatura externa	Atenção	42 °C; ainda segura, mas próxima do limite superior
Integridade	Normal	Estrutura íntegra
Energia	Normal	Acima de 80%
Pressão	Normal	Dentro da faixa
Módulos críticos	Normal	Todos com status OK

Não foram identificadas anomalias críticas no cenário inicial. Como riscos de acompanhamento, a IA pode sugerir monitorar aumento de temperatura, queda rápida de energia, variação de pressão e mudanças nos módulos críticos. Em sistemas reais, a IA deve atuar como apoio e não como único mecanismo responsável pela decisão.

1.6 Reflexão crítica

Ética e responsabilidade. Sistemas de alta criticidade exigem critérios verificáveis, supervisão humana e responsabilidade sobre as decisões. Uma falha de software pode produzir consequências humanas, financeiras e ambientais.

Impacto social. A exploração espacial contribui para comunicação, navegação, previsão do tempo, ciência e monitoramento ambiental. Ao mesmo tempo, envolve custos elevados e desafios como a geração de lixo espacial.

Sustentabilidade tecnológica. Projetos devem buscar eficiência energética, reutilização, maior vida útil dos equipamentos e redução de resíduos, considerando o ciclo de vida completo da tecnologia.

Conclusão

O projeto demonstrou a organização e a verificação de dados de telemetria, a implementação de um algoritmo em Python, a análise energética e o uso de IA como apoio à identificação de riscos. No cenário simulado, o resultado final foi PRONTO PARA DECOLAR.

Repositório público no GitHub:

<https://github.com/pedrocastrovs/Miss-o-Aurora-Siger>